

React Interview Prep

Topic 1 — What is React & Why Does It Exist?

BEGINNER → INTERMEDIATE → EXPERT

■ Topic	What is React & Virtual DOM
■ Level	Beginner to Expert
■ Series	React Interview Prep — Session 1

by Akshay Chavhan • React Interview Series

1. What is React?

React is a **JavaScript library** (not a framework) built by Facebook in 2013 to solve one problem:

"Efficiently update the UI when data changes — without re-rendering everything."

■ Analogy — Like You're 5

Old websites = LEGO city where you demolish everything to change one building.

React = each building is its own independent LEGO block — change only that block.

■ ■ Two Core Ideas React Introduced

Concept	What It Means	Benefit
Component-Based Architecture	UI = Tree of small, reusable, independent pieces	Reusability, isolation, easy maintenance
Virtual DOM	Lightweight JS copy of real DOM; diffs old vs new	Only changed parts update → fast UI

■ Component Example

```
function Avatar({ name }) {  
  return <img src={`/avatars/${name}.png`} alt={name} />;  
}
```

```
function UserCard({ name, role }) {  
  return (  
    <div>  
      <Avatar name={name} /> {/* reusable! */}  
      <h2>{name}</h2>  
    </div>  
  );  
}
```

2. Virtual DOM — How It Works



JSX Object (Virtual DOM Node)

```
console.log(<h1 className='title'>Hello</h1>);  
  
// outputs:  
{  
  type: 'h1',
```

```
props: { className: 'title', children: 'Hello' },
$$typeof: Symbol(react.element) // ← Virtual DOM node
}
```

3. Declarative vs Imperative

Aspect	Imperative (Old Way)	Declarative (React)
You describe...	HOW to change the DOM step by step	WHAT the UI should look like
DOM control	You manually call DOM APIs	React handles DOM internally
Example tool	jQuery / vanilla JS	React state + JSX
Maintainability	Hard — complex for large apps	Easy — UI = f(state)

```
// ■ Imperative (jQuery way)
document.querySelector('h1').innerText = 'Hi';
document.querySelector('h1').style.color = 'red';

// ■ Declarative (React way)
const [color, setColor] = useState('black');
return <h1 style={{ color }}>Hi</h1>;
// Just change state → React handles the DOM automatically
```

4. Expert — React Fiber & Reconciliation

React's engine under the hood is split into two parts:

Reconciler	Decides WHAT changed (pausable, prioritized units)	Renderer	Decides HOW to apply it react-dom (web) / react-native (mobile) / react-three-fiber (3D)
-------------------	--	-----------------	--

Key insight: React Fiber (v16+) made rendering interruptible — the browser can handle high-priority tasks mid-render. This enables Concurrent Mode, Suspense, and transitions.

5. Prove It Locally — Quick Setup

```
npm create vite@latest react-proof -- --template react
cd react-proof && npm install && npm run dev
```

What to Prove	Where to Look	What You See
Component Tree	DevTools → Components tab	App > UserCard > Avatar tree

Virtual DOM diffing	DevTools → Elements panel	Only changed node flashes yellow
JSX = JS object	Browser Console → console.log()	{ type, props, \$\$typeof } object
Reconciliation	DevTools → Profiler → Record	Only re-rendered components highlighted
Declarative	setState vs direct DOM call	React auto-syncs UI from state

■ Interview Q&A; — Topic 1

Q1. What is React and why was it built?

→ React is a JavaScript library for building UIs using components. Facebook built it to solve the problem of efficiently updating only the parts of the UI that change, instead of re-rendering the whole page.

Q2. Is React a framework or a library?

→ It's a library — it only handles the View layer. You compose your own stack: React Router for routing, Redux/Zustand for state, TanStack Query for data. Angular is a full framework by contrast.

Q3. What is the Virtual DOM and why does it exist?

→ Virtual DOM is a lightweight JS object tree mirroring the real DOM. When state changes, React creates a new Virtual DOM, diffs it against the old one (Reconciliation), and patches only the changed nodes in the real DOM — making updates fast and cheap.

Q4. What is Reconciliation?

→ Reconciliation is React's algorithm for comparing previous and new Virtual DOM trees to find the minimum number of real DOM operations needed. React's Fiber architecture makes this process interruptible and prioritized.

Q5. What is the difference between declarative and imperative UI?

→ Imperative = you tell the DOM HOW to change step by step (e.g., `querySelector + innerText`). Declarative = you describe WHAT the UI should look like for a given state, React figures out HOW to sync the DOM.

Q6. Can one file have multiple components?

→ Yes. A file is just a container. Convention: 1 file = 1 exported component, but small private sub-components tightly coupled to the main one can live together in the same file.